

Phase 5 – Apex Programming (Developer)

Doctor Appointment & Consultation System (Healthcare CRM)

Objective

To extend the automation capabilities of the Doctor Appointment & Consultation System by using Apex programming for custom business logic that cannot be achieved through declarative tools. This phase introduces triggers, classes, and SOQL queries to handle data operations, validations, and notifications efficiently.

Apex Components Implemented

1. Apex Trigger – Auto Prescription Creation After Appointment Completion

Purpose:

Automatically create a Prescription record when an Appointment status changes to “Completed”. This ensures that once a consultation ends, the system prepares a ready prescription record linked to the patient and doctor.

Trigger Name: AppointmentTrigger

Trigger Event: After Update

Code Logic (Example):

```
trigger AppointmentTrigger on Appointment__c (after update) {  
  
    List<Prescription__c> presList = new List<Prescription__c>();  
  
    for(Appointment__c app : Trigger.new){  
  
        Appointment__c oldApp = Trigger.oldMap.get(app.Id);  
  
        // Check if appointment status changed from Scheduled to Completed  
  
        if(app.Status__c == 'Completed' && oldApp.Status__c != 'Completed'){  
  
            Prescription__c pres = new Prescription__c();  
  
            pres.Appointment__c = app.Id;  
  
            pres.Issued_By__c = app.Doctor__c;
```

```

        pres.Prescription_Date__c = System.today();

        pres.Approval_Status__c = 'Pending';

        presList.add(pres);
    }
}

if(!presList.isEmpty()){
    insert presList;
}
}

```

Explanation:

- Trigger runs **after update** on Appointment.
- Checks if status has changed to **Completed**.
- Creates a **new Prescription** automatically, linked to the same Appointment and Doctor.
- Inserts the record into the Prescription object.

Impact:

This automates post-consultation workflows and saves time for doctors by pre-generating a prescription form.

2. Apex Class – Doctor Utility (SOQL & Collections)

Purpose:

Provide reusable methods to retrieve doctor and appointment information using SOQL queries.

Class Name: DoctorUtility

Code:

```

public class DoctorUtility {

    // Fetch all appointments for a specific doctor

    public static List<Appointment__c> getDoctorAppointments(Id doctorId) {

```

```

List<Appointment__c> apps = [
    SELECT Id, Patient__r.Name, Appointment_Date__c, Status__c
    FROM Appointment__c
    WHERE Doctor__c = :doctorId
    ORDER BY Appointment_Date__c DESC
];

return apps;
}

// Count completed appointments

public static Integer getCompletedAppointments(Id doctorId) {
    return [SELECT COUNT() FROM Appointment__c WHERE Doctor__c = :doctorId AND Status__c =
'Completed'];
}
}

```

Explanation:

- Demonstrates **SOQL** to retrieve appointment data.
- Uses **List** and **Integer** collections.
- The class can be reused in Lightning components or future automation.

3. Apex Trigger Design Pattern

Purpose:

To maintain scalability and best practices, the project follows the **Trigger Handler Pattern** for clean code separation.

Example:

```

trigger AppointmentTriggerHandler on Appointment__c (after update) {
    AppointmentTriggerHandlerClass.afterUpdate(Trigger.new, Trigger.oldMap);
}

```

```

public class AppointmentTriggerHandlerClass {

    public static void afterUpdate(List<Appointment__c> newList, Map<Id, Appointment__c> oldMap){

        List<Prescription__c> presList = new List<Prescription__c>();

        for(Appointment__c app : newList){

            Appointment__c oldApp = oldMap.get(app.Id);

            if(app.Status__c == 'Completed' && oldApp.Status__c != 'Completed'){

                Prescription__c pres = new Prescription__c(

                    Appointment__c = app.Id,

                    Issued_By__c = app.Doctor__c,

                    Prescription_Date__c = Date.today(),

                    Approval_Status__c = 'Pending'

                );

                presList.add(pres);

            }

        }

        if(!presList.isEmpty()){

            insert presList;

        }

    }

}

```

Benefit:

- Keeps logic outside the trigger for cleaner, reusable, and testable code.
- Simplifies future maintenance when additional logic is added.

4. Apex Test Class

Purpose:

To verify the correctness of the trigger and class logic with code coverage for deployment.

Code Example:

@isTest

```
public class AppointmentTriggerTest {
```

```
    @isTest
```

```
    static void testPrescriptionCreation() {
```

```
        // Create a Doctor User
```

```
        User doc = [SELECT Id FROM User WHERE Profile.Name = 'System Administrator' LIMIT 1];
```

```
        // Create a Patient and Appointment
```

```
        Appointment__c app = new Appointment__c(
```

```
            Doctor__c = doc.Id,
```

```
            Patient__c = null,
```

```
            Appointment_Date__c = Date.today(),
```

```
            Status__c = 'Scheduled'
```

```
        );
```

```
        insert app;
```

```
        // Update Appointment status to Completed
```

```
        app.Status__c = 'Completed';
```

```
        update app;
```

```
        // Verify Prescription created
```

```
        List<Prescription__c> presList = [SELECT Id, Appointment__c FROM Prescription__c WHERE  
Appointment__c = :app.Id];
```

```
        System.assertEquals(1, presList.size(), 'Prescription should be created after appointment completion');
```

```
    }
```

}